

UW PICO 5.09

File: io_redirection.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_ARGS 64
#define BUF_SIZE 1024

// Trim surrounding whitespace from strings
char *trim_space(char *str) {
    while (*str == ' ' || *str == '\t') str++;
    if (*str == 0) return str;
    char *end = str + strlen(str) - 1;
    while (end > str && (*end == ' ' || *end == '\t')) end--;
    end[1] = '\0';
    return str;
}

void execute_redirection(char *line) {
    char *input_file = NULL;
    char *output_file = NULL;
    int append_mode = 0;

    // 1. Parse Output Redirection (>> or >)
    char *out_append_pos = strstr(line, ">>");
    char *out_trunc_pos = strchr(line, '>');

    if (out_append_pos) {
        append_mode = 1;
        *out_append_pos = '\0';
        output_file = trim_space(out_append_pos + 2);
    } else if (out_trunc_pos) {
        append_mode = 0;
        *out_trunc_pos = '\0';
        output_file = trim_space(out_trunc_pos + 1);
    }

    // 2. Parse Input Redirection (<)
    char *in_pos = strchr(line, '<');
    if (in_pos) {
        *in_pos = '\0';
        input_file = trim_space(in_pos + 1);
    }

    // Clean base command string
    char *cmd_str = trim_space(line);
```

^G Get Help
^X Exit

^O WriteOut
^J Justify

^R Read File
^W Where is

^Y Prev Pg
^V Next Pg

^K Cut Text
^U UnCut Text

UW PICO 5.09

File: io_redirection.c

```
// 1. Parse Output Redirection (>> or >)
char *out_append_pos = strstr(line, ">>");
char *out_trunc_pos = strchr(line, '>');

if (out_append_pos) {
    append_mode = 1;
    *out_append_pos = '\0';
    output_file = trim_space(out_append_pos + 2);
} else if (out_trunc_pos) {
    append_mode = 0;
    *out_trunc_pos = '\0';
    output_file = trim_space(out_trunc_pos + 1);
}

// 2. Parse Input Redirection (<)
char *in_pos = strchr(line, '<');
if (in_pos) {
    *in_pos = '\0';
    input_file = trim_space(in_pos + 1);
}

// Clean base command string
char *cmd_str = trim_space(line);

// Tokenize base command arguments
char *args[MAX_ARGS];
int arg_count = 0;
char *token = strtok(cmd_str, " \t");
while (token != NULL && arg_count < MAX_ARGS - 1) {
    args[arg_count++] = token;
    token = strtok(NULL, " \t");
}
args[arg_count] = NULL;

if (arg_count == 0) return;

// Save original stdin/stdout descriptors to restore stream later
int saved_stdin = dup(STDIN_FILENO);
int saved_stdout = dup(STDOUT_FILENO);

pid_t pid = fork();

if (pid < 0) {
    perror("fork failed");
} else if (pid == 0) {
    // Child Process: Handle Input Redirection (<)
    if (input_file && strlen(input_file) > 0) {
```

^G Get Help
^X Exit

^O WriteOut
^J Justify

^R Read File
^W Where is

^Y Prev Pg
^V Next Pg

^K Cut Text
^U UnCut Text

-- -zsh

...

...memory_and_test_shell.c

-- nano signal_handler.c

-- -zsh

-- appen

UW PICO 5.09

File: io_redirection.c

```
// Tokenize base command arguments
char *args[MAX_ARGS];
int arg_count = 0;
char *token = strtok(cmd_str, " \\t");
while (token != NULL && arg_count < MAX_ARGS - 1) {
    args[arg_count++] = token;
    token = strtok(NULL, " \\t");
}
args[arg_count] = NULL;

if (arg_count == 0) return;

// Save original stdin/stdout descriptors to restore stream later
int saved_stdin = dup(STDIN_FILENO);
int saved_stdout = dup(STDOUT_FILENO);

pid_t pid = fork();

if (pid < 0) {
    perror("fork failed");
} else if (pid == 0) {
    // Child Process: Handle Input Redirection (<)
    if (input_file && strlen(input_file) > 0) {
        int in_fd = open(input_file, O_RDONLY);
        if (in_fd < 0) {
            perror("Input redirection error (Missing file)");
            exit(EXIT_FAILURE);
        }
        dup2(in_fd, STDIN_FILENO);
        close(in_fd);
    }

    // Child Process: Handle Output Redirection (> or >>)
    if (output_file && strlen(output_file) > 0) {
        int flags = O_WRONLY | O_CREAT | (append_mode ? O_APPEND : O_TRUNC);
        int out_fd = open(output_file, flags, 0644);
        if (out_fd < 0) {
            perror("Output redirection error (Permission/Path issue)");
            exit(EXIT_FAILURE);
        }
        dup2(out_fd, STDOUT_FILENO);
        close(out_fd);
    }

    execvp(args[0], args);
    perror("Command execution failed");
    exit(EXIT_FAILURE);
}
```

^G Get Help
^X Exit

^O WriteOut
^J Justify

^R Read File
^W Where is

^Y Prev Pg
^V Next Pg

^K Cut Text
^U UnCut Tex

-- -zsh

...

...memory_and_test_shell.c

-- nano signal_handler.c

-- -zsh

-- appen

UW PICO 5.09

File: io_redirection.c

```
int in_fd = open(input_file, O_RDONLY);
if (in_fd < 0) {
    perror("Input redirection error (Missing file)");
    exit(EXIT_FAILURE);
}
dup2(in_fd, STDIN_FILENO);
close(in_fd);

// Child Process: Handle Output Redirection (> or >>)
if (output_file && strlen(output_file) > 0) {
    int flags = O_WRONLY | O_CREAT | (append_mode ? O_APPEND : O_TRUNC);
    int out_fd = open(output_file, flags, 0644);
    if (out_fd < 0) {
        perror("Output redirection error (Permission/Path issue)");
        exit(EXIT_FAILURE);
    }
    dup2(out_fd, STDOUT_FILENO);
    close(out_fd);
}

execvp(args[0], args);
perror("Command execution failed");
exit(EXIT_FAILURE);
} else {
    // Parent Process: Wait for Child to complete
    int status;
    waitpid(pid, &status, 0);

    // Restore Streams back to terminal standard descriptors
    dup2(saved_stdin, STDIN_FILENO);
    dup2(saved_stdout, STDOUT_FILENO);
    close(saved_stdin);
    close(saved_stdout);
}

int main() {
    char input_buf[BUF_SIZE];

    while (1) {
        printf("\nRedirection Shell > ");
        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

        input_buf[strcspn(input_buf, "\n")] = 0; // Strip newline
        if (strcmp(input_buf, "exit") == 0) break;
        if (strlen(input_buf) == 0) continue;
    }
}
```

^G Get Help
^X Exit

^O WriteOut
^J Justify

^R Read File
^W Where is

^Y Prev Pg
^V Next Pg

^K Cut Text
^U UnCut Tex

```
~ -- zsh      ...      ...memory_and_test_shell.c      ~ -- nano signal_handler.c      ~ -- zsh      ~ -- appen

UW PICO 5.09                                     File: io_redirection.c

} else {
    // Parent Process: Wait for Child to complete
    int status;
    waitpid(pid, &status, 0);

    // Restore Streams back to terminal standard descriptors
    dup2(saved_stdin, STDIN_FILENO);
    dup2(saved_stdout, STDOUT_FILENO);
    close(saved_stdin);
    close(saved_stdout);
}
}

int main() {
    char input_buf[BUF_SIZE];

    while (1) {
        printf("\nRedirection Shell > ");
        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

        input_buf[strcspn(input_buf, "\n")] = 0; // Strip newline
        if (strcmp(input_buf, "exit") == 0) break;
        if (strlen(input_buf) == 0) continue;

        execute_redirection(input_buf);
    }

    return 0;
}

^G Get Help      ^O WriteOut      ^R Read File      ^Y Prev Pg      ^K Cut Text
^X Exit          ^J Justify       ^W Where is      ^V Next Pg      ^U UnCut Tex
```

```
medikondarakesh@Medikondas-MacBook-Air ~ % nano io_redirection.c
medikondarakesh@Medikondas-MacBook-Air ~ % gcc -g -Wall io_redirection.c -o io_redirection
./io_redirection
```

```
Redirection Shell > echo Hello File Redirection > output.txt
```

```
Redirection Shell > echo Second Line >> output.txt
```

```
Redirection Shell > grep Hello < output.txt
Hello File Redirection
```

```
Redirection Shell > cat < nonexistent_input_file.txt
Input redirection error (Missing file): No such file or directory
```

```
Redirection Shell > exit
```